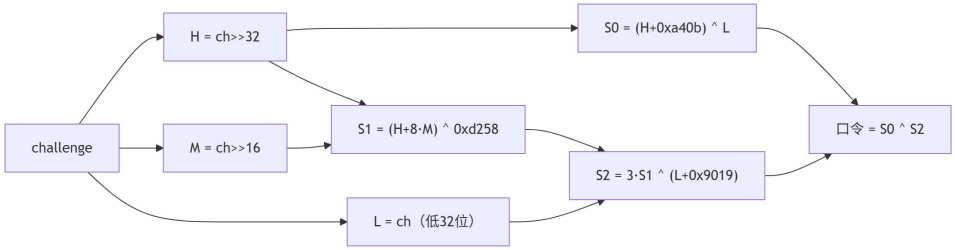
Bomblab 实验报告

一、关键汇编、计算关系和推理过程

Bomb1 函数调用、参数传递、返回值



关键汇编：三个 step 各一次「加 + 异或」，p1_compute 把首尾结果异或返回，phase_01 与输入比较。

计算关系:（H = ch >> 32, M = ch >> 16, L = ch）

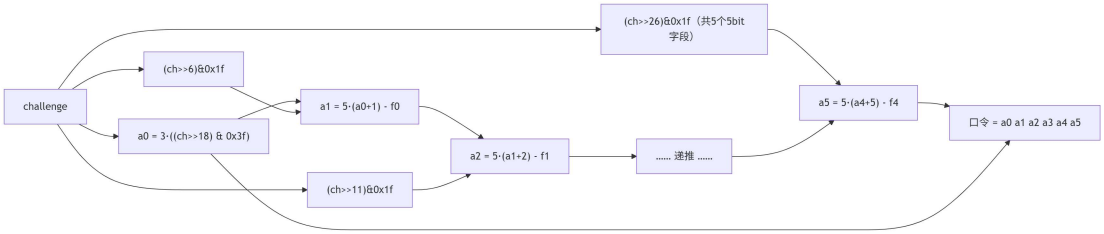
S0 = (H + 0xa40b) ^ L

S1 = (H + 8 * M) ^ 0xd258

S2 = (3 * S1) ^ (L + 0x9019)

推理：三个 step 的中间量 S0/S1 不直接输出，只有 p1_compute 末尾的 S0 ^ S2 进 eax，被 phase 拿来和输入比较，所以口令只需算这一个值。seed 42 → 4015594384。

Bomb2 数组、循环、递推关系



关键汇编：phase_02 用 "%u %u %u %u %u" 读 6 个数；先验 arr[0]，再一个 5 次循环，循环体 imul $5 → sub（减数来自 mov %ch,%rax; shr %cl; and $0x1f），cl 每轮 + 5。

计算关系:

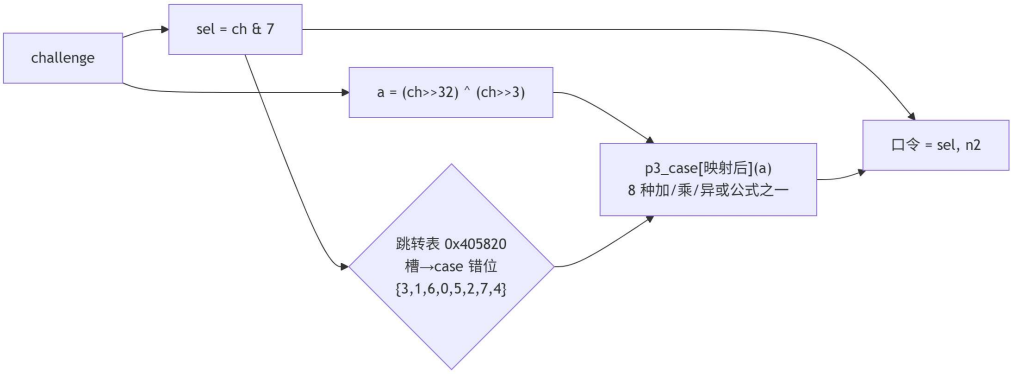
arr[0] = 3 * ((ch >> 18) & 0x3f)

arr[i] = 5 * (arr[i - 1] + i) - ((ch >> cl) & 0x1f), i = 1..5

cl = 6, 11, 16, 21, 26

推理：后一个数只依赖前一个数 → 是递推；challenge 中每 5 位取一个 5 - bit 字段当扰动，移位量 6 起步、每轮加 5，正好取 5 个不重叠字段。seed 42 → 9 21 96 480 2396 12001。

Bomb3 switch 分支与跳转表



关键汇编：读 2 个数；p3_dispatch 里 mov -0x4(%rbp),%eax; and $7; call *0x405820(,%rax,8)，第二个数与返回值 cmp。

计算关系:

n1 = ch & 7

a = (ch >> 3) ^ (ch >> 32)

n2 = case_映射[n1](a)

槽位 → case: {3,1,6,0,5,2,7,4}

case0 (a^0xb75e) + (a >> 2) case4 (a^0x7fdf) + (a >> 1)

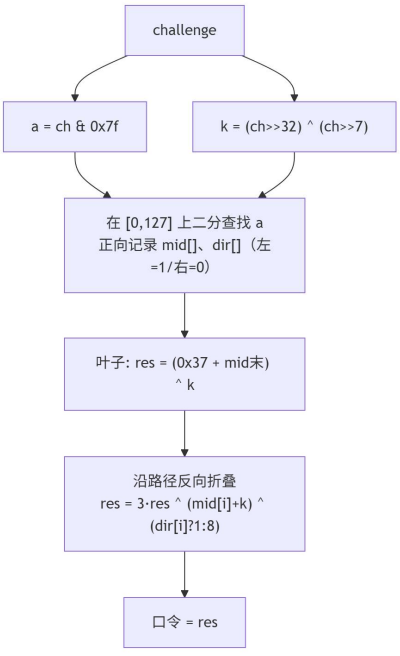
case1 5a + 0x6cd6 case5 4a ^ 0x7089

case2 5a + 0x3f3f9 case6 5a + 0x785a

case3 4a ^ 0x64f1 case7 5a + 0xd746

推理：跳转表里存的是函数地址不是 case 号，把 8 个表项地址逐个反查到 p3_case_N 符号，才发现槽位和 case 错位；第一个口令就是槽位号本身（ch&7），第二个才是函数返回值。seed 42（槽 4 → case5） → 4 3881207485。

Bomb4 递归调用与返回过程



关键汇编（p4_walk）：取中点 m = ((hi - lo) >> 1) + lo，cmp 相等就走到叶子；否则按大小把 lo/hi 收到 m ± 1 并递归；回溯时 imul $3、xor (mid + k)，左/右分支分别再 xor $1 / xor $8，叶子

是 (0x37 + mid)^k。

计算关系:

a = ch & 0x7f, k = (ch >> 7) ^ (ch >> 32)

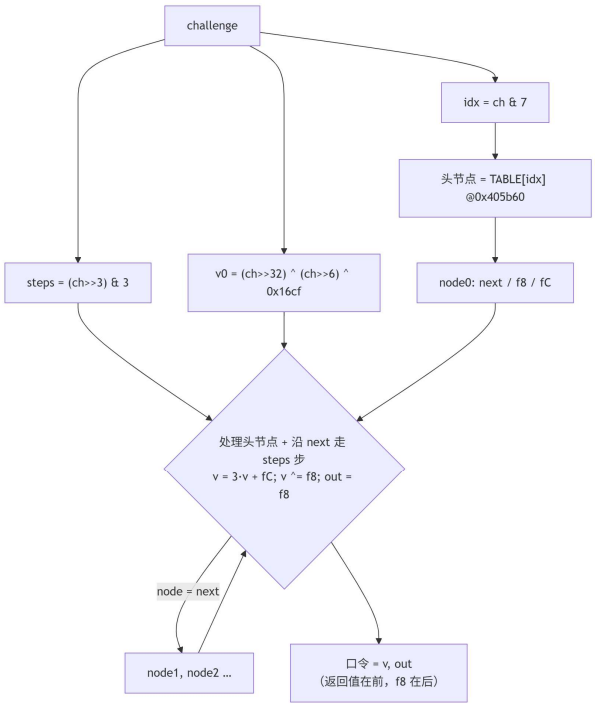
在 [0,127] 上二分 a，记录每步 mid[] 和方向 dir[]（左 = 1, 右 = 0）

叶子: res = (0x37 + mid[末]) ^ k

回溯: res = 3 * res ^ (mid[i] + k) ^ (dir[i] ? 0x1 : 0x8)

推理：本质是"记录二分搜索路径，再沿路径折叠"。区间只有 128、深度 ≤ 8，所以先正向把路径存进数组，再从叶子反向回代，全部逻辑留在一个函数内。seed 42 → 1434899252。

Bomb5 结构体布局、指针、链表遍历



关键汇编:

mov 0x405b60(, %rax, 8), %rdx # 头 = TABLE[ch & 7]

mov 0x8(%rdx), %esi # esi = node -> f8

add 0xc(%rdx), %eax # v += node -> fC

xor %esi, %eax # v ^= f8

mov (%rdx), %rdx # node = node -> next

cmp %edx, - 0x1c(%rbp)

cmp %eax, - 0x18(%rbp)

计算关系:

steps = (ch >> 3) & 3

v = (ch >> 6) ^ (ch >> 32) ^ 0x16cf

对头节点及之后 steps 个 next（共 steps + 1 个节点）:

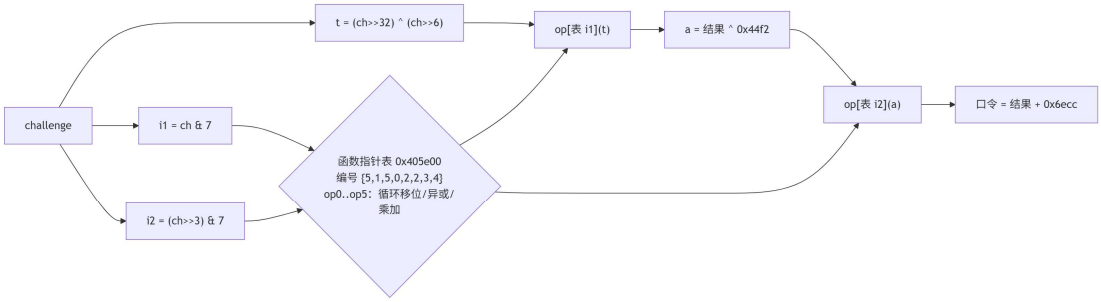
v = 3 * v + fC; v ^= f8; out = f8; node = next

口令 = v, out

推理：结构体三个字段的偏移直接从 0x8 / 0xc / 0x0 读出；循环次数为 0 时只处理头节点。sscanf 第 1 个输出（栈上低地址 - 0x1c）拿去和返回值比，不是和 * out 比，所以必须 v out。

seed 42（idx = 6, steps = 0） → 2753780401 1581

Bomb6 函数指针表与间接调用



关键汇编:

t = (ch >> 6) ^ (ch >> 32)

call * 0x405e00(,%rax,8) # a = op[ch & 7](t)

xor $0x44f2,%eax

and $0x38,%ebx # (ch >> 3)&7 换成字节偏移

call * 0x405e00(%rbx) # b = op[(ch >> 3)&7](a)

add $0x6ecc,%eax

计算关系:

t = (ch >> 6) ^ (ch >> 32)

a = op[ch & 7](t) ^ 0x44f2

口令 = op[(ch >> 3) & 7](a) + 0x6ecc

表 0x405e00 → 编号 {5,1,5,0,2,2,3,4}

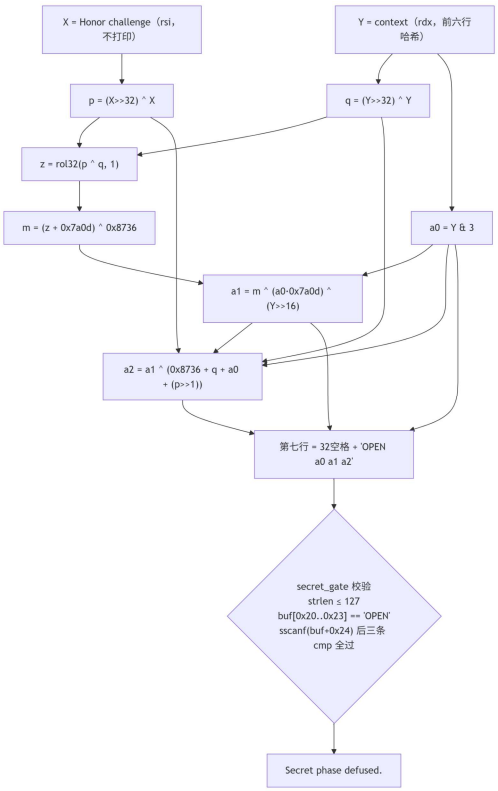
op0 ror32(x,5) op3 (0x5c72 + 4x) ^ 3x

op1 rol32(x,1) ^ 0x4ed3 op4 (x + 0xc008) ^ (x >> 5)

op2 (x^0x9571) + (x << 7) op5 (x^(x >> 2)) + 0x7812

推理：两次间接调用的下标不同（ch&7 与 (ch >> 3)&7），中间固定插一个 xor 0x44f2；把 0x405e00 的 8 个地址反查到 p6_op_N，得到编号表（注意也有重复和错位），再把每个 op 函数的移位/异或翻成 C。seed 42（op2 → op0） → 1445938385。

Honor



关键汇编:

secret_gate（0x402750）：call strlen 后限长；cmpl $0x4e45504f, -0xb0(%rbp)（即缓冲偏移 0x20 处 **4** 字节=小端 "OPEN"）；lea -0xac(%rbp), %rdi 把偏移 0x24 传给 secret_phase。

secret_phase（0x402670）：sscanf(buf + 0x24, "%u %u %u", &a0, &a1, &a2)，然后三条 cmp 校验。

计算关系（X = challenge, Y = context 由 defuser 传入）

p = (X >> 32)^X, q = (Y >> 32)^Y

a0 = Y & 3

a1 = ((rol32(p^q, 1) + 0x7a0d) ^ 0x8736) ^ (a0 * 0x7a0d) ^ (Y >> 16)

a2 = a1 ^ (0x8736 + q + a0 + (p >> 1))

推理：魔数 0x4e45504f 按字节拆就是 "OPEN"（小端），它要求出现在固定偏移 **0x20**，所以前面垫 **32** 个空格；后面的三个十进制数由 X、Y 唯一确定，直接计算。困难之处在于自由文本不是整数行，得用 snprintf 拼且 challenge 不打印，要在 secret_gate 入口用 GDB 读 rsi(= X) / rdx(= Y)。seed 42 → OPEN 3 **2584953969 2871703937**。

二、seed 验证记录

使用非公开 seed 888 对第 1 至 6 关进行了验证，均成功输出 Phase N defused.

三、运行结果截图

```
python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

```
终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

四、对实验的意见和建议

实验感受：

本次 Bomb Lab 是一次极具挑战性的逆向工程体验。与以往编写代码的实验不同，这次实验要求我们面对没有源码的二进制文件，通过阅读汇编指令、结合 GDB 动态调试来“拆解”未知的逻辑。在实验初期，我深刻体会到了底层汇编与高级语言之间的巨大鸿沟，尤其是在处理寄存器截断（如 32 位与 64 位之间的溢出）、复杂的嵌套调用和跳转表时，任何一点直觉上的误差都会导致“BOOM”。

通过这次实验，我在短时间内熟练掌握了 GDB 的核心技巧，更深刻理解了 C 语言底层的函数运作机制。当最终看到屏幕上亮起所有 [SAFE]与 Secret phase defused 时，那种凭借纯粹的技术手段“黑”进程序的成就感是无可比拟的。

对助教的建议：建议在实验指导书中，提前强调一下 GDB 调试时 p/u 与有符号/无符号类型转换的坑，因为手动计算 32 位截断时容易翻车。在推导涉及复杂位运算的关卡时，最大的困难是 C 语言中的隐式类型提升与汇编寄存器截断之间的冲突（例如 uint32_t 与 uint64_t 在乘法或加法时的溢出行为）。建议在课件中提供一个小示例，演示强制转换模拟汇编中 %eax 的 32 位回绕截断。最后感谢助教在环境配置和自动评测上做的工作。